Trajectory-Based Drift Detection in Stratified Agent Memory

A Minimal Deterministic Note on Coherence Decay Across Time

Stephen A. Putman
putmanmodel@pm.me

Version 0.2 — March 2026

Abstract

Stratified agent architectures provide mechanisms for controlled memory formation, deviation handling, and constraint-grounded inference. However, such systems are often evaluated primarily at discrete time steps, leaving a gap between local admissibility and time-extended coherence. A system may remain stepwise valid while gradually departing from its prior internal trajectory. This note introduces a minimal method for detecting such drift by treating memory updates as a trajectory over time and evaluating that trajectory with simple, inspectable metrics. Using update magnitude, cross-tier divergence, and directional break, we show how coherence decay can be made visible in a deterministic micro-run without introducing any new architectural subsystem. The result is a compact, replayable set of drift signals computed from already-existing stratified states and update relations.

1. Introduction

Constraint-governed agent architectures are designed to preserve disciplined state transitions under bounded observability. In the surrounding stack, memory stratification distinguishes short-term volatility from longer-horizon persistence, while deviation governance detects structurally relevant departures from stratified baselines. Yet most such evaluations remain local: they ask whether a given event, signal, or update is admissible at a given step.

This leaves a gap in typical evaluation practice. A system may remain locally consistent while gradually drifting away from its prior direction of development. No single update need appear catastrophic. No individual step need violate a boundary on its own. Nevertheless, across time, the relation between short-horizon accumulation and deeper structure may begin to deform. The result is not merely a bad event, but a weakening of internal coherence.

This note proposes a minimal extension: treat memory updates themselves as a trajectory and measure whether that trajectory remains coherent. The aim is narrow. We do not introduce a new cognition layer, a new ontology of failure, or a free-floating interpretive narrative. We instead isolate a measurable trajectory signal that can be computed from already-existing stratified memory and governed update dynamics.

2. Stratified Memory and Update Trajectories

We assume a three-tier memory model:

- m(t): short-term state

- u(t): intermediate state
- a(t): structural or long-horizon state

This follows the stratified memory formulation in which short-term state captures volatile transitions, intermediate state compresses recurrent regularity, and structural state changes slowly under higher write resistance. Identity is not treated as a static flag, but as coherence across these coupled tiers over time. For the purposes of this note, assume tier states are vectors in a shared representation space, and let Δx(t) denote the step-to-step change in state x at time t.

Each tier evolves through discrete updates:

- $\Delta m(t)$
- $\Delta u(t)$
- $\Delta a(t)$

Stratification imposes asymmetry on these updates. Surface state responds rapidly to new input. Intermediate state smooths and accumulates. Structural state lags, resists transient perturbation, and shifts only under sustained pressure. This lag is not a defect; it is part of what preserves continuity. But once lag becomes misalignment, it may become observable as drift.

For the purposes of this note, we define an update trajectory as the time-ordered sequence of tier updates:

$$T = \{(\Delta m(t), \Delta u(t), \Delta a(t))\}_t$$

We define the trajectory over all tiers, but the minimal demo metrics below use Δu(t) and u(t) - a(t) as practical projections of that trajectory. In practice, the intermediate tier is the most useful anchor for a minimal demo because it is where short-horizon variation begins to compress into a more stable representation while still remaining measurable step by step. This keeps the formulation minimal while matching the broader stratified logic.

3. Drift as Trajectory Deviation

We define trajectory drift as deviation in the path of memory updates across time, especially when recent accumulation begins to separate from longer-horizon structure or when update direction breaks sharply relative to its prior course.

Drift is not identical to any single contradiction or anomaly. It is not merely a surprising event. It emerges through a time-extended pattern such as:

- accumulation of directional change
- growing divergence between intermediate and structural state
- a sharp break in update direction after a period of apparent continuity

This matters because systems can remain locally admissible while nevertheless becoming less coherent as ongoing processes. In that sense, drift is not a narrative label pasted onto behavior after the fact. It is a measurable deformation in the update path of an already-governed system.

4. Minimal Drift Metrics

We use three simple metrics, each evaluated at every timestep.

4.1 Update Magnitude

Update magnitude measures the size of the current intermediate-tier change:

$$\|\Delta u(t)\|$$

This captures local update strength. Large values do not, by themselves, imply drift. They indicate that the system is moving strongly at that step. Their relevance depends on whether the movement remains aligned with prior direction and deeper structure.

4.2 Cross-Tier Divergence

Cross-tier divergence measures mismatch between recent accumulation and longer-horizon structure:

$$\|u(t) - a(t)\|$$

This operationalizes a familiar stratified condition: short- and medium-horizon updates may move faster than structural priors, but excessive or sustained mismatch indicates lag turning into misalignment.

4.3 Directional Break

Directional break measures change in update direction from one step to the next:

$$cos(\Delta u(t), \Delta u(t-1))$$

High positive values indicate continuity of direction. Values near zero indicate a break or redirection. Negative values indicate reversal.

This metric is intentionally simple. It does not claim to capture all meaningful arc structure. Its role is narrower: it identifies whether the current update remains pointed in roughly the same direction as the preceding update, or whether the internal path has fractured.

Taken together, these three metrics provide a minimal and inspectable signal of trajectory coherence: how strongly the system is moving, how far recent accumulation has separated from deeper structure, and whether the direction of movement itself remains continuous.

5. Deterministic Micro-Run

To evaluate these metrics, we use a deterministic scripted run with four phases:

- stable
- gradual shift
- contradiction
- recovery

The purpose of the micro-run is not to simulate full agent intelligence. It is to isolate one architectural claim: a system can appear locally well-behaved at the step level while developing a detectable trajectory-level coherence problem across time.

The run is fully scripted and contains no stochastic elements. Because the event sequence and updates are fixed, any observed shift in metric behavior is attributable to the scripted event sequence and governed update dynamics rather than to randomness in extraction, retrieval, or routing. That makes the result suitable as a minimal proof-of-behavior rather than an intuition-only illustration.

In the present run, the early timesteps remain stable: directional continuity is high and cross-tier divergence remains modest. A gradual bend then appears before the formal contradiction point, indicating that directional continuity begins to decline before the contradiction phase is entered. The contradiction begins at $t = 10$. At that point, directional continuity drops sharply while update strength rises, indicating that the system is still moving, but no longer along its prior internal line. Cross-tier divergence then continues to grow and reaches its local peak at $t = 12$, showing that mismatch between recent accumulation and structural lag has become most pronounced. The strongest directional break occurs at $t = 13$, where the directional metric becomes strongly negative, indicating reversal rather than mere redirection. Recovery then follows as divergence falls and directional continuity stabilizes again.

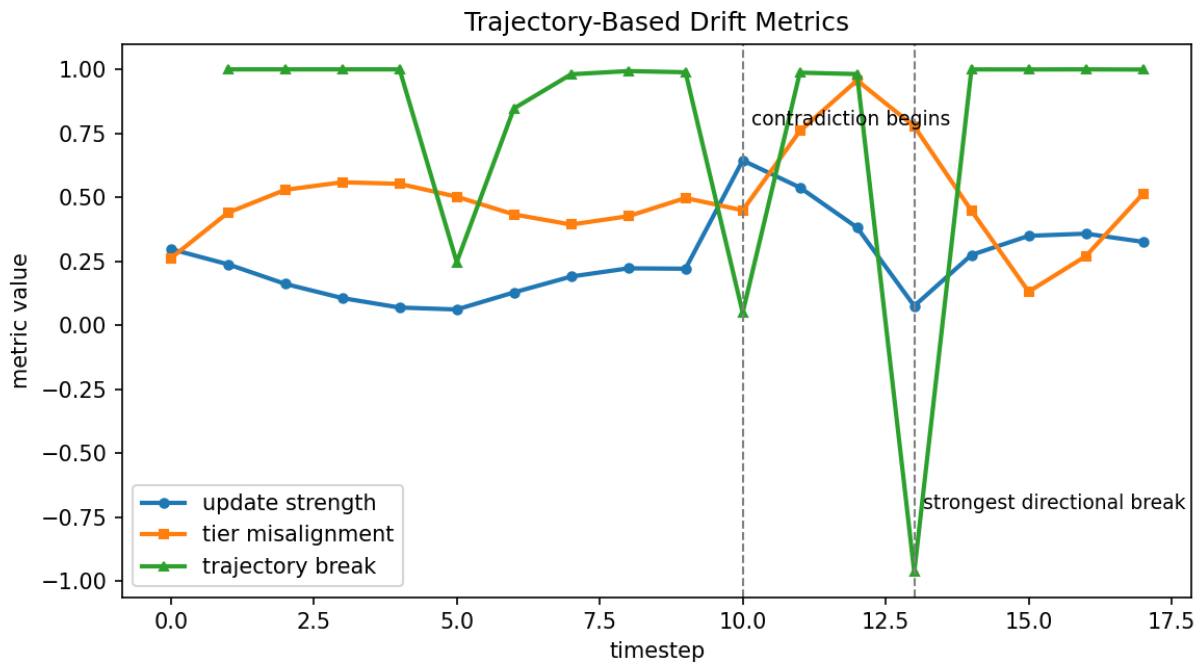

Figure 1. Drift metrics across the deterministic micro-run.
Update strength, cross-tier divergence, and directional break across timesteps. Contradiction begins at t = 10*, divergence peaks at* t = 12*, and the strongest directional break occurs at* t = 13*.*

That sequence matters because the three metrics are not redundant. The contradiction onset is first visible as a directional problem. Misalignment peaks slightly later as a cross-tier problem. The largest reversal then appears during the turn out of contradiction. Drift here is therefore not one scalar spike, but a small structured pattern across related observables.

6. Observations from the Run

The deterministic trace supports three basic observations.

First, in this run, the contradiction phase is scripted, but its onset is still detectable through measurable trajectory signatures rather than post hoc narrative description alone. In the trace, contradiction onset appears as a local collapse in directional continuity at t = 10 while the system is still actively updating. This is exactly the sort of turning-point behavior that per-step validation can miss if it does not evaluate the path across time.

Second, cross-tier divergence lags the directional disturbance rather than coinciding with it perfectly. This is important. In a stratified system, deeper structure is supposed to lag. The issue is not lag itself, but the point at which lag becomes sustained misalignment. The run makes that visible by showing divergence peaking after contradiction begins rather than at the exact instant of first break.

Third, recovery is not simply the absence of error. It has its own recognizable signature. In the run, the strongest directional reversal occurs during the turn out of contradiction rather than at the moment contradiction begins. The system has to bend back. Recovery is therefore visible as a real trajectory event, not just a return to silence.

These observations are modest by design. They do not prove a universal theory of instability. They show that trajectory-level coherence decay, misalignment, and recovery can be detected in a compact deterministic setting using metrics computed from already-existing tier states and their relations.

7. Discussion

The contribution of this note is narrow but useful.

It does not add a new architectural layer. It does not replace deviation governance. It does not authorize hidden-state storytelling. It simply changes the object of evaluation from isolated update events to the update path itself. In that sense, the proposal is closer to a readout than to a subsystem: a measurable trajectory signal computed from already-existing tier dynamics and made inspectable under deterministic replay.

This framing also clarifies the relation to neighboring papers. Memory Stratification provides the persistence logic: short-term volatility, intermediate accumulation, structural lag, and identity as cross-tier coherence over time. Constraint Deviation Engine provides the language of inspectable deviation, turning-point detection, persistence rules, hysteresis, and reviewable routing. This note sits between them. It asks what becomes visible when the object under evaluation is not just the current deviation vector, but the temporal behavior of tiered updates themselves.

It is also important to state what this note does not claim. It does not claim intent, belief, selfhood, or hidden psychological states. It does not claim that every bend in trajectory is pathological. It does not claim that update reversal is always bad. And it does not relax any existing boundary around persistence or promotion. Trajectory signals may make drift more visible, but they do not by themselves authorize promotion, baseline mutation, or durable state adjustment. Those remain governed transitions.

8. Conclusion

This note presents a minimal method for detecting internal drift in stratified agent systems by evaluating the trajectory of memory updates across time. Instead of asking only whether a given step is locally admissible, it asks whether the path formed by those steps remains coherent.

Using three simple metrics—update magnitude, cross-tier divergence, and directional break—and a deterministic micro-run, we show how coherence decay can be made visible without introducing any new subsystem. In the present run, contradiction begins at $t = 10$, divergence peaks at $t = 12$, and the strongest directional break occurs at $t = 13$, illustrating that drift, misalignment, and recovery appear as distinct but related phases of one trajectory rather than as a single undifferentiated event.

The result is a compact and inspectable signal of drift grounded in existing stratified states, existing update relations, and deterministic replay. That is the point of the note: not to inflate the architecture, but to make one of its already-latent failure signatures easier to see.